

Incident Report — `my-app` Pod Stuck Pending: HPE CSI Provisioning Failure Caused by Cluster DNS Loss (Calico VXLAN Return-Path Fault)

Field	Value
Date (UTC)	2026-10-05 09:22 (PVC created) → 2026-10-06 01:55 (resolved) — outage ≈ 16 h 33 min
Cluster	RKE2 <code>v1.33.13+rke2r2</code> — control-plane <code>wna-vs vp-adv8m1/m2/m3</code> , workers <code>wna-vs vp-adv8w1/w2/w3</code> (Ubuntu 26.04)
CNI	Calico <code>v3.32.1</code> , VXLAN mode (<code>vxlan.calico</code> , UDP/4789) — typha enabled
DNS	RKE2 CoreDNS 1.14.6 (<code>rke2-coredns-rke2-coredns</code> , ClusterIP <code>10.43.0.10</code> , 2 pods: <code>10.42.63.2 @m2</code> , <code>10.42.162.210 @m3</code>)
Storage stack	HPE CSI Driver for Kubernetes <code>v3.3.0</code> , HPE CSP <code>v3.3.0</code> (Alletra 6000 / Nimble), iSCSI, SC <code>hpe-rwo</code>
Affected	<code>default/my-app</code> (PVC <code>app-data</code> 10 GiB stuck Pending ~16.5 h); latent impact on all cross-node pod traffic in the cluster
Status	✅ Resolved via workaround — pod <code>Running</code> , PV <code>Bound</code> , <code>/data</code> read/write verified. ⚠️ Underlay fault still open (see §8–§9)

1. Executive Summary

The pod `my-app-685d4d9d5d-rwbpb` stayed `Pending` for ~16.5 hours because its PVC `app-data` never bound. The HPE CSI controller's `CreateVolume` calls kept timing out because the driver could not log in to its backend (CSP):

```
Post "http://alletra6000-csp-svc:8080/containers/v1/tokens":
  dial tcp: lookup alletra6000-csp-svc: i/o timeout
```

Dual-node `tcpdump` capture proved that **CoreDNS itself is healthy** — queries reach it and it answers correctly (A `10.43.90.150`) — but in-cluster **DNS reply packets are lost in the cross-node return path of the Calico VXLAN overlay (UDP/4789)**. Any DNS lookup (and, in fact, any pod-to-pod traffic) whose return path crosses nodes fails; same-node and host-underlay traffic work normally. With 2 CoreDNS replicas on 2 different nodes, roughly half of all cluster DNS traffic is affected.

Workaround applied (operator-approved): all DNS dependencies were removed from the storage path. Secret `hpe-backend` was patched to use IP addresses instead of names (CSP service → ClusterIP `10.43.90.150`; storage array FQDN → `192.0.2.10`), then `hpe-csi-controller` was restarted. The volume provisioned within seconds; the pod scheduled and started; the iSCSI volume at `/data` was verified read/write.

The VXLAN underlay fault itself is NOT fixed. It must be addressed (MTU/PMTUD, checksum offload, underlay firewall for UDP/4789, or Calico dataplane reset) or other workloads will keep

failing intermittently. See §8 and §9.

2. Symptoms

Pod

```
$ kubectl get pods -n default
NAME                                READY   STATUS    RESTARTS   AGE
my-app-685d4d9d5d-rwbpb           0/1     Pending   0           95s    # 16.5 h by diagnosis time

$ kubectl describe pod my-app-685d4d9d5d-rwbpb -n default | tail -4
Events:
  Warning   FailedScheduling   default-scheduler
            0/6 nodes are available: pod has unbound immediate PersistentVolumeClaims.
            preemption: 0/6 nodes are available: 6 Preemption is not helpful for scheduling.
```

PVC

```
$ kubectl get pvc -n default
NAME      STATUS    VOLUME   CAPACITY   STORAGECLASS   AGE
app-data  Pending               hpe-rwo                      15h

$ kubectl describe pvc app-data -n default | tail -6
Events:
  Normal    ExternalProvisioning   Waiting for a volume to be created either by the external
                                provisioner 'csi.hpe.com' or manually by the system administrator ...
  Warning   ProvisioningFailed     rpc error: code = DeadlineExceeded desc = context deadline exceeded
```

Notes:

- StorageClass `hpe-rwo` uses `volumeBindingMode: Immediate` → provisioning must succeed **before** scheduling, which is exactly why the pod sat `Pending`.
- The pod spec itself was fine (`nginx:1.27-alpine`, 100m/128Mi requests, no `nodeSelector`) — the blocker was purely the unbound PVC.

3. Investigation Chain

Step 1 — Scheduler → PVC → external provisioner

Scheduler reports "unbound immediate PVC"; PVC events show `csi.hpe.com` timing out. See above.

Step 2 — `csi-provisioner` sidecar log (`hpe-csi-controller`)

```
GRPC call  /csi.v1.Controller/CreateVolume  name=pvc-c100e2b7-...  required_bytes=10737418240
GRPC response  err="rpc error: code = DeadlineExceeded desc = context deadline exceeded"  # repeats
every ~30 s
```

Generic 30-s deadline; the actual error is only visible in the driver container.

Red herring: sidecar logs `Retrying syncing claim key="c100e2b7-67eb-452f-b510-f41c1e9b8652"` (bare UID, no namespace/name) — legacy code path in external-provisioner, same PVC, not a fault.

Step 3 — `hpe-csi-driver` container log → first real error

```
Adding connection to CSP at IP array.example.com, port 8080,
  context path , with username <csp-username> and serviceName alletra6000-csp-svc
About to attempt login to CSP for backend array.example.com
ERROR Failed to login to CSP. Status code: 0. Error: Post
  "http://alletra6000-csp-svc:8080/containers/v1/tokens":
  dial tcp: lookup alletra6000-csp-svc: i/o timeout
ERROR Volume creation failed, err: ... Failed to get storage provider from secrets ...
```

Key insight: the driver builds the CSP URL from secret key `serviceName` and relies fully on **cluster DNS**. Status code: 0 + `i/o timeout` after 30 s = name resolution layer, **not** the service itself.

Step 4 — Is the CSP backend healthy?

Object	Value
<code>svc/hpe-storage/alletra6000-csp-svc</code>	ClusterIP <code>10.43.90.150</code> , <code>8080/TCP</code>
Endpoint	<code>10.42.134.109:8080</code> (<code>nimble-csp-5f6b488b89-dbcts</code> , <code>1/1</code> Running, same node as CSI controller: w2)

Connectivity test (from same node, ignoring DNS):

Test	Result
Call CSP by Service name	✗ bad address (DNS down)
<code>POST http://10.43.90.150:8080/containers/v1/tokens</code> (ClusterIP)	✓ HTTP 415 — service answered
<code>POST http://10.42.134.109:8080/...</code> (pod IP)	✓ HTTP 415

→ CSP, service, endpoints: all healthy. Problem isolated to **DNS**.

Step 5 — Scope of the DNS failure

CSI pod `resolv.conf`: `nameserver 10.43.0.10`, search `hpe-storage.svc.cluster.local svc.cluster.local cluster.local example.com`, `ndots:1` — correct. (RKE2 quirk: DNS service is named `rke2-coredns-rke2-coredns`, there is **no** `kube-dns` Service object.)

Test	From	Result
busybox pod → <code>10.43.0.10:53</code> (ClusterIP)	workers w1, w2, w3; cp m1	✗ timeout on every node
busybox pod → CoreDNS pod IPs <code>10.42.63.2:53</code> , <code>10.42.162.210:53</code> (UDP and TCP)	worker	✗ timeout

Test	From	Result
kubectl debug node/... (hostNetwork) → 10.43.0.10:53 and 10.42.63.2:53	w2	✗ timeout
Node host → external DNS 10.71.1.44:53 (registry-1.docker.io etc.)	w2, m2	✓ fast, correct answers
nc cross-node pod-IP connectivity (TCP 53 and other ports)	worker	✗ timeout, not refused (silent drop)
Same-node pod → ClusterIP / pod IP (any port)	w2	✓ works (HTTP 415 example)

→ Not "port 53 blocked": **all cross-node pod-network traffic fails meanwhile same-node traffic and host-underlay traffic work** — pointing to the VXLAN overlay data plane.

Step 6 — Kubernetes-layer causes eliminated (evidence kept for the record)

Suspect	Verdict	Evidence
NetworkPolicy	✗ not it	Only 1 networkpolicy in the whole cluster: cattle-fleet-system/default-allow-all ; Calico GlobalNetworkPolicy / networkpolicies.crd.projectcalico.org / staged variants: none
CoreDNS RBAC	✗ stale log noise	Pod logs showed system:serviceaccount:kube-system:coredns cannot list services/namespaces/endpointslices — historical only. kubectl auth can-i list <...> --as=system:serviceaccount:kube-system:coredns → yes for all; CoreDNS demonstrably had full caches (Step 7)
kube-proxy rules	✗ correct	KUBE-SERVICES → KUBE-SVC-PUNXDRXNIM3ELMDM (tcp/53) and KUBE-SVC-YFPH5LFNKP7E3G4L (udp/53) → KUBE-SEP-* DNAT to both CoreDNS pods 50/50 present; KUBE-PROXY-FIREWALL / KUBE-NODEPORTS / KUBE-EXTERNAL-SERVICES empty; KUBE-FIREWALL only standard loopback rule
Calico policy chains	✗ stock	cali-INPUT / cali-FORWARD / cali-from-wl-dispatch contain only default Felix rules; no port-53 DROP; INPUT / FORWARD policies ACCEPT
Calico control plane	✗ healthy	Felix reconcile loops avg 12-22 ms; typha watches normal; calico-node up
CoreDNS pods / Endpoints	✗ healthy	Both Running, EndpointSlice rke2-coredns-rke2-coredns-qj9hx contains both pod IPs

Step 7 — Decisive evidence: paired tcpdump on two nodes

Setup: privileged nsenter root pods on **w2** (client node) and **m2** (hosts CoreDNS 10.42.63.2),
tcpdump -i any -n "port 53" on both, while a DNS-probe pod on **w2** queried alletra6000-csp-svc.hpe-storage.svc.cluster.local .

On m2 (CoreDNS node) — query arrives, answered CORRECTLY, reply emitted:

```
01:33:33.964035 vxlan.calico      In  10.42.134.111.36561 > 10.42.63.2.53: 6437+ A? alletra6000-csp-
svc.hpe-storage.svc.cluster.local.
01:33:33.964111 calidbf90bc97ab Out 10.42.134.111.36561 > 10.42.63.2.53: 6437+ A? ...client...
01:33:33.964545 calidbf90bc97ab In  10.42.63.2.53 > 10.42.134.111.36561: 6437*- 1/0/0 A 10.43.90.150
← correct answer for the CSP service
```

```
01:33:33.964553 vxlan.calico      Out 10.42.63.2.53 > 10.42.134.111.36561: 6437*- 1/0/0 A 10.43.90.150
← reply forwarded onto the overlay
```

On w2 (client node) — every query leaves; no reply ever returns:

```
01:34:22.401359 calif2806c5b037 In  10.42.134.112.49934 > 10.43.0.10.53: 28569+ A? alletra6000-csp-
svc... (DNAT by kube-proxy, correct)
01:34:22.401422 vxlan.calico      Out 10.42.134.112.49934 > 10.42.162.210.53: 28569+ ...
##### no "In" vxlan.calico packets from 10.42.63.2 / 10.42.162.210 EVER appear
#####
01:34:24.904045 (same query retried) ... → client eventually gives up: "connection timed out; no
servers could be reached"
```

Interpretation:

- Outbound path: pod → cali* → kube-proxy DNAT → vxlan.calico = fully functional.
- Return path: reply **leaves** m2's vxlan.calico but **never arrives** on w2 → the loss is in the **VXLAN underlay transport between hosts (UDP/4789) or its acceptance on the receiving node**.
- Related Calico rule pair (both nodes, standard Felix template): -A cali-INPUT -p udp --dports 4789 ... --match-set cali40all-vxlan-net src ... -j ACCEPT # VXLAN from known nodes -A cali-INPUT -p udp --dports 4789 ... -j DROP # all other VXLAN An empty/stale cali40all-vxlan-net ipset would produce exactly this symptom. The ipset binary is not installed on the Ubuntu hosts and could not be dumped during the session — flagged as a follow-up diagnostic (§9).
- Additional corroboration that this is infrastructure-wide, not DNS-specific: cross-node bare nc to pod IPs times out on arbitrary ports (silent drop, not refused); meanwhile CoreDNS pods on m3 log their **own** outbound/upstream problems (read udp 10.42.162.210:...->10.71.1.44:53: i/o timeout at times) indicating broader overlay/egress flakiness around the time of the incident.

Step 8 — Why the failure looked "total" for CSI but random for everything else

kube-dns has 2 replicas pinned to 2 different nodes; with connection-oriented client retries and a broken reply path for (at least) some node pairs, lookups become a coin-flip. The CSI path retried forever and always hit dead paths; short-lived --rm test pods simply reported bad address/timed out.

4. Root Cause

Primary (infrastructure — STILL OPEN): Cross-node **Calico VXLAN (UDP/4789) return-path packet loss** in the overlay underlay path. Reply packets emitted on the destination node's vxlan.calico never reach the origin node. CoreDNS consumers therefore see lookup ...: i/o timeout for every query whose reply crosses nodes. Precise physical cause (MTU/PMTUD blackhole, VXLAN checksum/UXO offload bug, underlay switch/firewall dropping UDP/4789 in one direction, or stale VXLAN FDB/ipset state after the earlier ~16 h-ago node disturbances — note the cluster-wide one-time pod restarts "16h ago" visible in kubectl get pods) **could not be reached via the Kubernetes API alone**.

Contributing design factor #1 (CSI): The HPE CSI driver logs in to CSP via

`http://<serviceName>:8080` (from secret `hpe-backend`), making dynamic storage provisioning fully dependent on cluster DNS — no fallback.

Contributing design factor #2 (CSP): After factor #1 was worked around, `nimble-csp` failed with:

```
Caused by: java.net.UnknownHostException: array.example.com
```

i.e. **the CSP application itself resolves the storage-array FQDN through cluster DNS as well.** A second dependency on the same broken service, hidden one layer deeper. (Array FQDN resolves externally to `192.0.2.10`; TCP 443 to it verified OPEN from the host.)

Everything else was exonerated with evidence: PVC/SC/pod specs, RBAC (live can-i checks), NetworkPolicies (none), kube-proxy DNAT, Felix dataplane reconcile, CSP app health, array reachability by IP.

5. Fix Applied (Workaround — approved: "keep the storage path from depending on cross-node DNS")

Note: the literal "pin the CSI controller next to CSP and CoreDNS" variant was infeasible: CoreDNS runs only on tainted control-plane nodes and CSP is fixed to w2 (+3 dual mode would still hit m3's replica 50% of the time). Therefore the **equivalent and stricter** measure was taken: **remove DNS from the storage path entirely.**

Actions

1. Backed up/inspected secret structure (values not printed): keys `backend`, `password`, `serviceName`, `servicePort`, `username` in secret `hpe-backend` (ns `hpe-storage`), consumed by SC `hpe-rwo` for all 4 secret hooks (provision/node-stage/node-publish/expand).

2. Patched secret — IP literals instead of names:

```
```console # CSP service name -> its ClusterIP (stable; changes only if the Service is
deleted/recreated) kubectl patch secret hpe-backend -n hpe-storage --type merge \ -p '{"data":
{"serviceName":"","$(printf '10.43.90.150' | base64)"}'```
```

```
Array management FQDN -> its IP kubectl patch secret hpe-backend -n hpe-storage --type
merge \ -p '{"data":{"backend":"","$(printf '192.0.2.10' | base64)"}'```
```

3. Restarted the controller (out-of-band `provisioner` recreates its CSP credentials cache on new CreateVolume calls):

```
console kubectl rollout restart deploy/hpe-csi-controller -n hpe-storage
```

## Verification

```
driver log (fixed):
Adding connection to CSP at IP 192.0.2.10, ... serviceName 10.43.90.150 # no more DNS lookups

events (2026-10-06 01:55:49Z):
ProvisioningSucceeded persistentvolumeclaim/app-data Successfully provisioned volume pvc-c100e2b7-67eb-452f-b510-f41c1e9b8652

$ kubectl get pvc app-data -n default
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS
AGE
app-data Bound pvc-c100e2b7-67eb-452f-b510-f41c1e9b8652 10Gi RWO hpe-rwo
16h

$ kubectl get pod -n default -l app=my-app -o wide
first instance after bind (as originally reported):
my-app-685d4d9d5d-rwbpb 1/1 Running 10.42.134.116 wna-vsvp-adv8w2
```

Volume on the array: 10 GiB, iSCSI, ext4, pool default, targetScope=group, perf policy default, PV pvc-c100e2b7-67eb-452f-b510-f41c1e9b8652 created 2026-10-06T01:55:49Z.

Functional I/O test (exec in the pod):

```
$ kubectl exec deploy/my-app -- sh -c 'df -h /data; echo hello > /data/test.txt; cat /data/test.txt'
/dev/mapper/mpatha 9.7G 2.0M 9.2G 0% /data
hello → WRITE OK
```

## Post-fix observation (expected RWO churn, no data loss)

A replacement pod ( my-app-685d4d9d5d-lhzrm, node w3, IP 10.42.158.91 ) rolled ~7 min later (deployment object was re-applied/restarted outside this session's scope). Transient FailedAttachVolume (Multi-Attach error ... already used by pod(s) ...-rwbpb) for ~12 s while the old pod terminated, then SuccessfulAttachVolume on w3 and container started. Verified /data/test.txt (content test, 5 B, preserved) — data intact, volume healthy on the new node (detach/attach across nodes worked correctly through the CSI path).

## Cleanup performed

All diagnostic pods removed: node-shell-2f1adbc2-... (w2), node-shell-m2 (m2), and all --rm test pods auto-deleted. Cluster left with only the fix (secret patch + normal controller restart).

## 6. What Still Works vs What Is Broken (current state of the overlay fault)

Path	Status	Basis
Host → external world / external DNS ( 10.71.1.44:53 )	✓	tcpdump: fast answers
Same-node pod → pod / ClusterIP / host	✓	HTTP 415 CSP, volume I/O

Path	Status	Basis
iSCSI data path pod ↔ array ( 10.16.x ↔ 192.0.2.10 , routed host underlay)	✓	mpatha mounted, writes OK
<b>Cross-node pod→pod (all protocols/tests done)</b>	✗	replies never arrive; nc timeouts
<b>Cross-node DNS (kube-dns name or pod-IP)</b>	✗	dual tcpdump: answer emitted, never received

Consequence: any **new** workload whose DNS answer or traffic return path crosses nodes will hang/timeout. Current `my-app` works *because its storage path is IP-pinned and nginx itself makes no DNS lookups*.

## 7. Timeline

Time (UTC)	Event
10-01 10:56	Cluster/RKE2 addons installed (Calico, CoreDNS...)
10-05 09:22:37	PVC <code>app-data</code> created; provisioning fails immediately (first <code>ProvisioningFailed</code> )
10-05 ~09:22→10-06 01:2x	~16.5 h of retry loop: <code>CreateVolume DeadlineExceeded</code> , pod Pending; (multiple system pods show a single restart ≈16 h ago ≈ 10-05 09:0x — probable time the underlay fault started)
10-06 01:14	<code>hpe-csi-driver</code> Helm release v1 reinstall/upgrade (controller 9/9 Running again) — retries continue failing
10-06 01:2x-01:30	triage steps 1-6; discovery: CSP reachable by IP, DNS cluster-wide broken, RBAC healthy, no policies
10-06 01:33-01:34	<b>dual tcpdump proves reply path lost</b> (§3 Step 7)
10-06 ~01:40	operator decides: workaround path = de-DNS the storage chain
10-06 01:4x	secret <code>hpe-backend.serviceName</code> → 10.43.90.150 ; controller restart; new failure surfaces <b>inside CSP</b> : <code>UnknownHostException: array.example.com</code>
10-06 01:5x	array FQDN resolved from host (→ 192.0.2.10 ); TCP 443 OPEN; secret <code>hpe-backend.backend</code> → 192.0.2.10 ; controller restarted
10-06 <b>01:55:49</b>	<code>ProvisioningSucceeded</code> ; PV bound; pod <code>rwbp</code> scheduled to w2, 1/1 Running ; <code>/data</code> write verified
10-06 ~02:02	pod rolled ( <code>lhzm</code> on w3); volume detach/attach across nodes succeeded; data verified
10-06 02:0x	diagnostic node-shell pods deleted; incident closed with residual risk §8

## 8. Residual Risks

- Overlay fault not fixed (H).** New pods with cross-node DNS/traffic will hang; problem will be attributed "randomly" to whatever component hits it first (as it was here → CSI).
- ClusterIP dependency.** If `hpe-storage/alletra6000-csp-svc` is deleted/recreated (e.g. a fresh Helm install of the CSI driver chart), its ClusterIP may change; secret `hpe-backend.serviceName` must then be re-patched. Safer long-term alternative: set `kubernetes.io/service.*` nothing —

instead make the **Service use a fixed** `spec.clusterIP`, or fix DNS so the name works again and revert the secret.

3. **Array IP dependency.** `192.0.2.10` must match the array management-group IP (currently confirmed; low change frequency). If the group IP changes, patch the secret.
4. **Node identity recovery node-stage/node-publish.** iSCSI discovery to the array happens over the routed storage net (works), but snapshot/expand operations also go through CSP→secret path — OK with the same IP-pinned secret.
5. **Helm drift.** The secret was patched outside Helm; a `helm upgrade/rollback` of the CSI release will **revert** it and re-break provisioning. Document the override (see §10 recipe).

---

## 9. Recommended Next Steps (to actually fix the overlay)

---

In order of likelihood/safety:

1. **ipset content check (run on each worker):** `bash ipset list cali40all-vxlan-net # must contain every node's PodCIDR (e.g. 10.42.63.0/24, 10.42.134.0/24, 10.42.162.0/24 ...) ipset list cali40all-hosts-net # must contain all node IPs 10.16.2.64-69` Missing entries ⇒ stale Felix dataplane ⇒ proceed to step 4.
2. **VXLAN/MTU audit:** `bash ip -d link show vxlan.calico # MTU (typically underlay MTU-50 for VXLAN)`  
`ip route get 10.42.63.2 # via vxlan.calico? mtu? ping -M do -s 1472 10.42.63.2 # from one node's pod`  
`netns/host - PMTU blackhole test ethtool -k ens33 | grep -i -E 'tx-checksum|generic-segmentation|udp.*fragmentation'` Try toggling `tx-checksum-udp / tx-udp-segmentation` if suspicious (classic VXLAN-corruption culprit on some NICs).
3. **Underlay capture, UDP/4789 both directions on the physical NIC:** `bash tcpdump -i ens33 -nn 'udp port 4789' -c 50 # run on m2 while w2 queries: is the encapped reply seen on the wire/arriving? Emitted-but-not-arrived ⇒ switch/router/firewall between nodes blocking/limiting UDP/4789 (or asymmetric routing). Wiring check needed ⇒ network team.`
4. **Calico dataplane reset (low risk, Kubernetes-side):** `bash kubectl rollout restart ds/calico-node -n calico-system` Fix old ipset/FDB/VXLAN state if underlay is clean.
5. **Review what changed ~16 h before** (2026-10-05 ≈ 09:00 UTC): the single synchronized pod restart wave across CoreDNS/Calico/system components suggests a node event (kernel/netfilter update, security policy reload, firewall push) at/near that moment. Check `journalctl -b` since that window, node package logs (`/var/log/dpkg.log`, unattended-upgrades), and any FW change ticket.
6. After the underlay fix: revert `hpe-backend` secret to use the service name and FQDN (or keep IPs but then harden #2), and re-run the verification in §10.

---

## 10. Verification / Re-check Recipes

---

```
end-to-end health (what we saw when healthy after the fix):
kubectl get pods -n default -l app=my-app -o wide # 1/1 Running
kubectl get pvc app-data -n default # Bound
kubectl exec -n hpe-storage deploy/hpe-csi-controller -c hpe-csi-driver -- \
 sh -c 'true' >/dev/null && echo controller-ok
kubectl logs -n hpe-storage deploy/hpe-csi-controller -c hpe-csi-driver --tail=50 \
```

```
| grep -E 'ERROR|Adding connection'
kubectl logs -n hpe-storage deploy/hpe-csi-controller -c csi-provisioner --tail=50
kubectl logs -n hpe-storage deploy/nimble-csp --tail=50 | grep -iE 'error|UnknownHost'
kubectl -n hpe-storage get svc alletra6000-csp-svc -o wide # record ClusterIP! keep secret in sync

DNS health (currently FAILING place: proves remaining overlay issue):
kubectl run dns --rm -i --restart=Never --image=busybox \
 -- nslookup kubernetes.default.svc.cluster.local # expect 10.43.0.10 answer; today: times out
```

---

## 11. Facts Reference (for the ticket)

---

```
Forwarded namespace pods relevant (hpe-storage):
hpe-csi-controller-... 9/9 Running node w2
 sidecars: csi-provisioner v6.3.0, csi-attacher v4.12.0, csi-snapshotter v8.6.0, csi-resizer v2.2.1,
 csi-extensions v1.3.0, volume-mutator v1.4.0, vg-snapshotter/vg-provisioner v1.1.0
 driver: quay.io/hpestorage/csi-driver:v3.3.0
nimble-csp-... 1/1 Running node w2 (10.42.134.109:8080)

SC hpe-rwo: provisioner csi.hpe.com | Delete | Immediate | expansion:true |
 params: accessProtocol=iscsi, fstype=ext4, secrets(hpe-backend @ hpe-storage x4 hooks),
 description="...ReadWriteOne"
PVC app-data: 10Gi RW0 Filesystem, created 2026-10-05T09:22:37Z
PV pvc-c100e2b7-67eb-452f-b510-f41c1e9b8652: created 2026-10-06T01:55:49Z,
 iSCSI/ext4, pool=default, perf=default, thick=false, encrypted=false, dedupe=true, targetScope=group

CoreDNS: pods 10.42.63.2@wna-vsvp-adv8m2, 10.42.162.210@wna-vsvp-adv8m3 (svc ClusterIP 10.43.0.10; RKE2
name rke2-coredns-rke2-coredns)
Array: array.example.com = 192.0.2.10 (TCP 443 OPEN from nodes), user <csp-username>
Nodes: w1 .67, w2 .68, w3 .69 | m1 .64, m2 .65, m3 .66 (underlay 10.16.2.0/24, NICs ens33;
external DNS 10.71.1.44)
Overlay: Calico VXLAN, vxlan.calico, UDP 4789, pod CIDRs per-node /24 under 10.42.0.0/16
```

**Bottom line:** the `my-app` outage was never a pod, PVC, StorageClass, Secret, CSP, or array problem. It was caused by the network underlay silently dropping packets inside Calico's VXLAN paths: every cross-node DNS lookup timed out, so the HPE CSI controller could not reach its backend. In-cluster storage provisioning now works only because its two DNS dependencies (CSP service name, array FQDN) were pinned to IP addresses. Until the Calico VXLAN fault is fixed at the node/underlay level, other workloads in this cluster will keep failing intermittently.